Name: Hubert Ofosu Asare
Pitt ID: 4690704

Fabric Client Site: UCSD
Fabric Server Site: WASH

Fabric Client IP address: 10.134.137.2
Fabric Server IP address: 10.133.4.2

Finding RTT

Average RTT: 65.516ms

Go-Back-N File Transfer Programs

GBN_Server.py Output:

```
ubuntu@server:~$ python3 GBN_Server.py -f copy_100KB.txt
Opened Server socket
Bound socket on port 12000
Packet 0        Bytes Recvd 1000        Time Elapsed 1.2159347534179688e-05     Throughput
657.9300392156863 Mbps
Got last packet with seq 101. Waiting 2.0 sec and then quitting
Final wait time expired. Exiting...
Bytes transferred: 100017
Time Elapsed: 7.016097545623779 sec
Throughput: 0.11404288420976656 Mbps
```

GBN_Client.py Output:

```
ubuntu@client:~$ python3 GBN_Client.py -a 10.133.4.2 -f test_file_100KB.txt
Final packet with seq 101 acknowledged. Time to quit...

=== Final Stats ===
Bytes transferred: 100017
Time Elapsed: 7.0853424072265625 sec
Throughput: 0.11292834615641387 Mbps
```

Window Size and Throughput

Expected throughput for window = 1:

How does your measurement from previous section compare to expectation?
With a window size = 1,
Expected Throughput = (1*1000*8/0.065516)/1000000 = 0.1221 Mbps
Which means my expected throughput is almost the same as the recorded throughput of 0.1129 Mbps in the previous section.

Window size needed for 1 Mbps throughput:
For a 1 Mbps throughput, Window size= (Throughput * RTT) / packet size
(1000000 * 0.065516) / (1000*8) = 8.1895 ~= 8 segments

GBN_Server.py Output with 1 Mbps window:
ubuntu@server:~$ python3 GBN_Server.py -f copy_1MB.txt
Opened Server socket
Bound socket on port 12000
Packet 0 Bytes Recvd 1000 Time Elapsed 9.5367431640625e-06 Throughput 838.8608 Mbps
Packet 1000 Bytes Recvd 1000007 Time Elapsed 8.678263902664185 Throughput 0.9218498180890791 Mbps
Got last packet with seq 1001. Waiting 2.0 sec and then quitting
Final wait time expired. Exiting...
Bytes transferred: 1000007
Time Elapsed: 8.678835153579712 sec
Throughput: 0.9217891408733879 Mbps

GBN_Client.py Output with 1 Mbps window:
ubuntu@client:~$ python3 GBN_Client.py -a 10.133.4.2 -f test_file_1MB.txt -w 8
Final packet with seq 1001 acknowledged. Time to quit...

=== Final Stats ===
Bytes transferred: 1000007
Time Elapsed: 8.74867558479309 sec
Throughput: 0.9144305240790573 Mbps

Comments:
The throughputs I recorded are really close. However, I think they are not exactly the same because both client and server elapsed times are recorded at slightly different times and so the computation of the throughput will also be slightly different since the elapsed time is a factor for this computation.

Loss Emulation

GBN_Server.py Output:
ubuntu@server:~$ python3 GBN_Server.py -f copy_1MB.txt
Opened Server socket
Bound socket on port 12000
Packet 0 Bytes Recvd 1000 Time Elapsed 1.0251998901367188e-05 Throughput 780.3356279069767 Mbps
Packet 1000 Bytes Recvd 1000007 Time Elapsed 18.853368997573853 Throughput 0.42433031470553023 Mbps
Got last packet with seq 1001. Waiting 2.0 sec and then quitting
Final wait time expired. Exiting...

Bytes transferred: 1000007
Time Elapsed: 18.853748321533203 sec
Throughput: 0.4243217774824645 Mbps

GBN_Client.py Output:
ubuntu@client:~$ python3 GBN_Client.py -a 10.133.4.2 -f test_file_1MB.txt -w 8
TIMEOUT: resending 8 packets. Last seq in-flight is 111
TIMEOUT: resending 8 packets. Last seq in-flight is 253
TIMEOUT: resending 8 packets. Last seq in-flight is 299
TIMEOUT: resending 8 packets. Last seq in-flight is 384
TIMEOUT: resending 8 packets. Last seq in-flight is 502
TIMEOUT: resending 8 packets. Last seq in-flight is 572
TIMEOUT: resending 8 packets. Last seq in-flight is 632
TIMEOUT: resending 8 packets. Last seq in-flight is 767
TIMEOUT: resending 8 packets. Last seq in-flight is 874
Final packet with seq 1001 acknowledged. Time to quit...

=== Final Stats ===
Bytes transferred: 1000007
Time Elapsed: 18.92600107192993 sec
Throughput: 0.42270186763675455 Mbps

Number of observed timeouts: 9
Number of expected timeouts: 10

Termination

The final timeout is set after the last packet is received thus, with this line of code:

server_socket.settimeout(final_timeout)

Why is final timeout needed?

This final timeout is needed to help the server exit gracefully without hanging up on the client should in case there is any acknowledgement or additional request that requires retransmission of the last packets. In this lab the client is sending a file to the server but in the case where the server is rather sending to the client, this timeout comes in handier because the client might have not received a packet and no acknowledgement calls for a retransmission of that lost packets. In the case where there is no last timeout, server hangs up on client would not be able to respond to such requests. As discussed in class, usually the server would have to wait for at least the RTT of the connection to get some kind of acknowledgement that everything went okey before hanging up.

What event leads to the sending client getting stuck?
In the event that the server has timed out and cannot respond to incoming requests from the client, the client keeps retransmitting previous packets taking it as negative acknowledgement NACK from the server.

Bonus question one
Attached files by name GBN_Server_mod.py and GBN_Client_mod.py are the modified code for server and client respectively. Added a new packet type TYPE_FILENAME to the util.py module. Before I go into the while loop to start sending the packets, I package and send the save as name to the server. I added an option for client to parse the save as name using '-s' command line flag else the client application sends a default value of "SaveAsFile.txt".
At the server side I set the name of the output file to none and when the first packet is received, I check for the header "TYPE_FILENAME" and set its data as the name of the output file.

Bonus question two (selective repeat)
SR_Server.py and SR_Client.py have selective repeat implementation. At the Server side, a packet buffer is initialized to store out of order data by their sequence numbers. After successfully receiving a packet and adding to the packet buffer in the main function, I send an ACK to the server and try to deliver data to application if it is in buffer and bears the next expected sequence number.
At the Client side, I check while there is still data to be transmitted or packets in the window before I start sending data. I try to pop out packets I have gotten ACKs from my window and delete such data from my 'sent packet' buffer. Then after timeout, I try resending packets I did not get ACKs before I timeout and close my socket.